

Pontificia universidad católica del Ecuador

PUCE VIRTUAL



INGENIERIA EN CIBERSEGURIDAD Y GESTION DE TI

QUINTO NIVEL

DESARROLLO EN PLATAFORMA

DOCENTE:

Damian Nicolalde

T1S3: Proyecto: análisis de plataforma digital y demostración cliente-servidor

AUTOR:

Alejandro Carriel

Segundo periodo ordinario

1. Identificación de la entrega

<i>Campo</i>	Detalle
<i>Asignatura</i>	Desarrollo en plataforma
<i>Tarea</i>	T1S3: Proyecto: análisis de plataforma digital y demostración cliente-servidor
<i>Docente</i>	Damian Nicolalde
<i>Autor</i>	Alejandro Carriel
<i>Prototipo</i>	SecureGuard — evolución de Semanas 1 y 2
<i>Servidor</i>	Node.js v26.4.0 · módulo nativo http · puerto 3000
<i>Rutas</i>	GET /api/salud · GET /api/incidentes · GET
<i>JSON</i>	/api/incidentes/:id · POST /api/incidentes

2. Justificación

Una plataforma digital no es una página aislada: es un sistema que combina interfaz, servicios, datos y roles para sostener un proceso a través de la red. En este trabajo se tomó SecureGuard, el prototipo de las Semanas 1 y 2, y se convirtió en una plataforma mínima con flujo cliente-servidor real. El objetivo de la Unidad 3 fue demostrar, con un entregable ejecutable, los conceptos de componente, servicio, rol, arquitectura n-capas, HTTP y un servidor Node.js que responde JSON.

Los componentes visibles son cuatro. El cliente (HTML, CSS y JavaScript organizado como MVC ligero) presenta métricas, listado, formulario y detalle. El servidor Node.js expone un contrato HTTP. El servicio de aplicación valida y orquesta. El repositorio persiste incidentes en un archivo JSON. Esa separación no es cosmética: si el almacenamiento pasara a una base de datos, solo cambiaría la capa de persistencia. El navegador seguiría haciendo GET /api/incidentes y recibiendo el mismo JSON.

Los servicios de la plataforma son deliberadamente pocos, porque el prototipo debe permanecer mínimo. GET /api/salud comprueba que el proceso está vivo. GET /api/incidentes entrega la colección. GET /api/incidentes/:id recupera un recurso. POST /api/incidentes registra un caso nuevo. Cada servicio tiene semántica HTTP: 200 para lectura exitosa, 201 para creación, 400 cuando la validación falla y 404 cuando el identificador no existe. En Semanas 1 y 2 el fetch leía incidentes.json como archivo estático; ahora apunta a la API y el alta ya no vive solo en la memoria del navegador.

Los roles se derivan de un centro de respuesta. El reportante carga el evento desde el formulario. El analista consulta, prioriza y abre el detalle. El administrador mantiene el servicio en el puerto 3000. El desarrollador trabaja por capas. En el entorno académico

estos roles se concentran en un solo estudiante, pero el diseño no los mezcla en un único archivo: la interfaz no guarda en disco y el repositorio no conoce HTML.

La arquitectura n-capas concreta esa idea. Presentación recibe la petición. Aplicación ejecuta el caso de uso y valida título, descripción, fecha, tipo, severidad y correo. Dominio construye la entidad y el identificador INC-xxxxxx-xxx. Persistencia lee o escribe el archivo. Infraestructura —el módulo http nativo de Node.js— solo sabe de sockets, rutas y tipos MIME. No se usó un framework para que el mecanismo HTTP quede explícito: método, ruta, cabeceras Accept y Content-Type, cuerpo y código de estado.

HTTP es el contrato entre cliente y servidor. Es un protocolo de petición y respuesta, sin estado entre llamadas. El navegador no entra al servidor: envía un mensaje y espera otro. Cada mensaje tiene método, recurso, cabeceras y, si aplica, un cuerpo. JSON es el formato de representación porque es ligero y natural en JavaScript tanto en el cliente como en Node.js. Esa homogeneidad permite que el modelo del front deje de fabricar objetos locales y consuma los que produce el back. Una misma ruta se puede abrir en el navegador, consultar con curl o invocar desde fetch. Durante las pruebas, un error 500 por un identificador mal escrito dejó claro que el servidor interpreta el código tal como está: mayúsculas, nombres exportados y rutas tienen que coincidir.

La justificación pedagógica es doble. Primero, se reutilizó una interfaz ya validada para mostrar evolución de plataforma y no un ejercicio desconectado. Segundo, el flujo quedó observable: el listado anuncia casos sincronizados vía HTTP; /api/incidentes muestra el JSON crudo; el formulario dispara un POST que persiste el registro, como se comprobó al crear los casos de ransomware y phishing y al abrir su detalle con ?id=. Esas evidencias demuestran que la Unidad 3 no se quedó en definiciones: materializó una plataforma digital mínima, coherente con ciberseguridad y lista para crecer hacia autenticación y persistencia más robusta.

3. Diagrama de componentes y roles

La plataforma SecureGuard funciona como canal de entrada para reportar incidentes de ciberseguridad y activar respuesta. El cliente nunca abre el archivo incidentes.json: solo habla HTTP con Node.js.

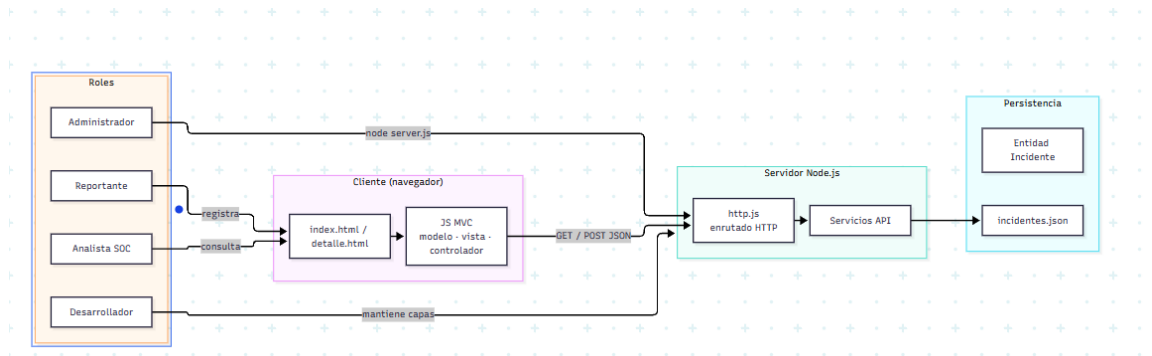


Figura 1. Diagrama de componentes: cliente, API Node.js, roles y persistencia.

Roles del prototipo: el reportante envía el caso (POST); el analista revisa el listado y el detalle (GET); el administrador ejecuta `node server.js` y consulta `/api/salud`; el desarrollador mantiene las fronteras entre capas.

4. Arquitectura n-capas

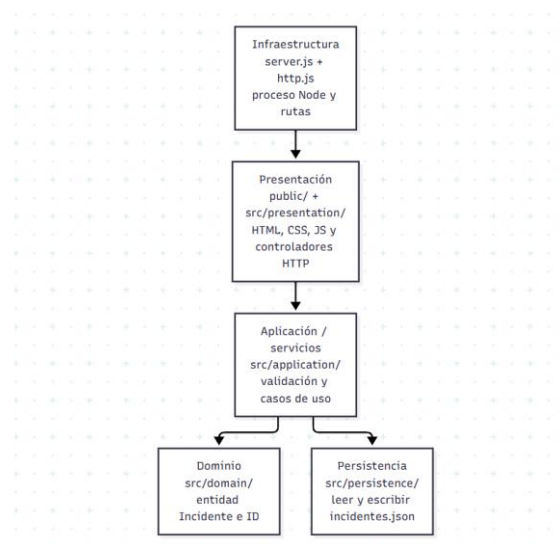


Figura 2. Arquitectura n-capas del prototipo.

Capa	Ubicación	Responsabilidad
Presentación	public/ y src/presentation/	UI HTML/CSS/JS y controladores HTTP
Aplicación	src/application/	Validación y casos de uso
Dominio	src/domain/	Entidad Incidente e identificador
Persistencia	src/persistence/	Lectura y escritura de incidentes.json
Infraestructura	server.js e http.js	Proceso Node y enrutado HTTP

5. Peticiones HTTP y servidor Node.js

Node.js ejecuta JavaScript fuera del navegador. El módulo http acepta la conexión, interpreta método y ruta, y escribe una respuesta con código de estado y cuerpo JSON en las rutas `/api/*`. El flujo de una lectura es: el navegador envía `GET /api/incidentes`; el controlador llama al servicio; el repositorio lee el archivo; el servidor responde HTTP 200 con `{ok, total, datos}`.

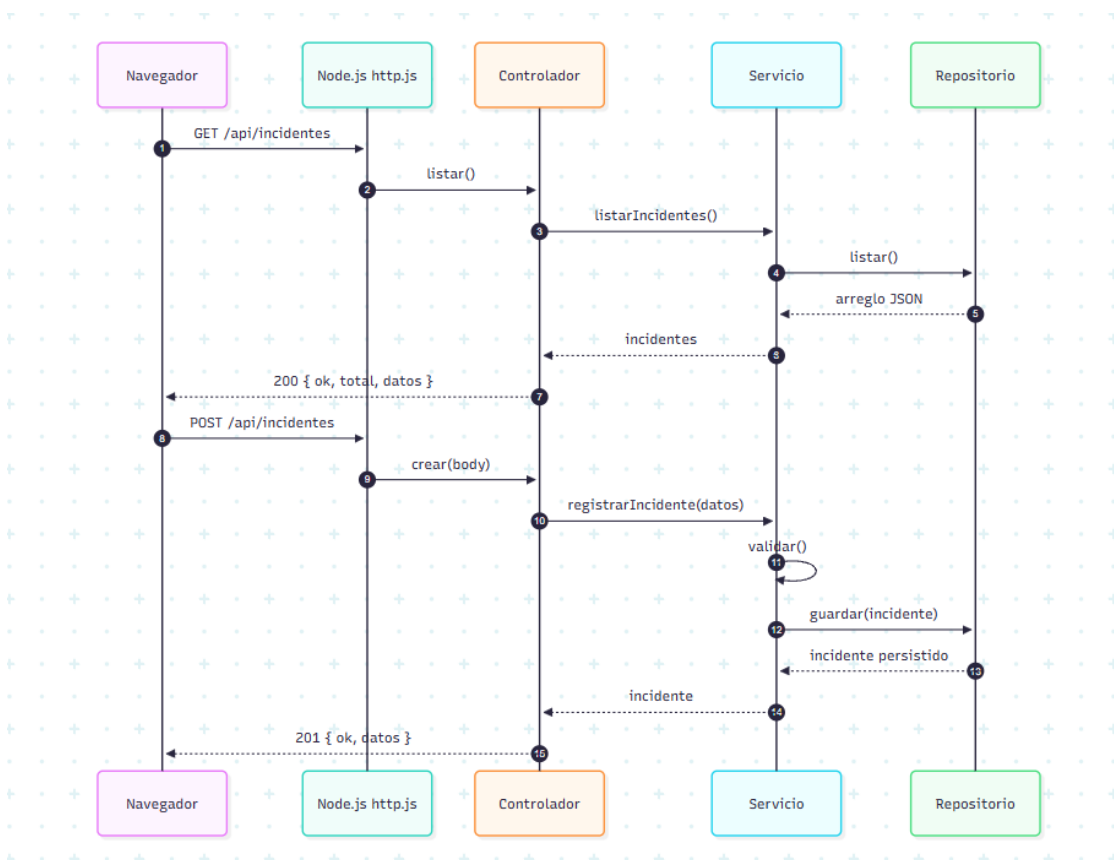


Figura 3. Secuencia HTTP de `GET /api/incidentes`.

6. Evidencias de funcionamiento

Las capturas se tomaron con el servidor en ejecución (node server.js) en `http://localhost:3000`. Se documenta el cliente, la persistencia de nuevos casos, la ruta JSON y el detalle dinámico.

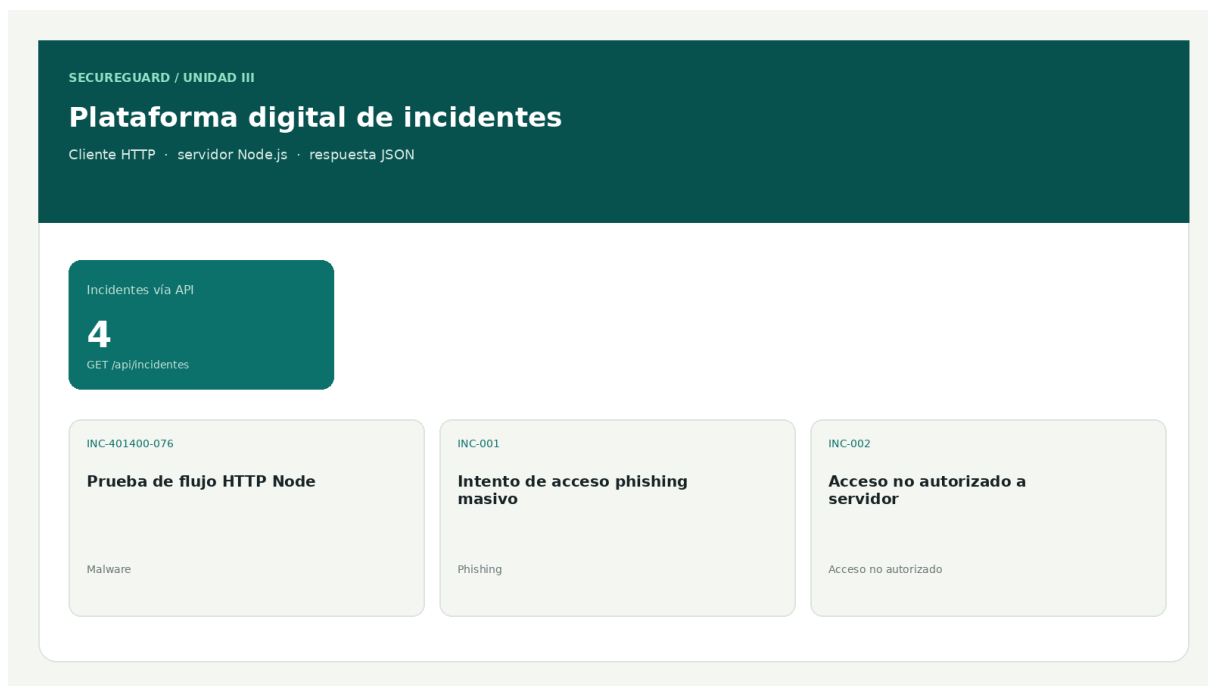


Figura 4. Cliente servido por Node.js en el puerto 3000.

Después de registrar casos desde el formulario, el listado pasó de 3 a 5 incidentes y el indicador cambió a «casos sincronizados vía HTTP». Los identificadores INC-364274-390 (Ransomware) e INC-122581-893 (posible Phishing) fueron generados por el dominio y persistidos en el JSON. Una recarga de la página los mantiene: ya no viven solo en memoria del navegador.

```
GET http://localhost:3000/api/incidentes

{"ok":true,"total":3,"datos":[{"id":"INC-001","titulo":"Intento de acceso phishing masivo","severidad":"alta","fecha":"2026-09-03","estado":"En investigación","tipo":"Phishing"}, {"id":"INC-002","titulo":"Acceso no autorizado a servidor","severidad":"crítica","fecha":"2026-09-02","estado":"Contención en curso","tipo":"Acceso no autorizado"}, {"id":"INC-003","titulo":"Vulnerabilidad detectada en aplicación interna","severidad":"media","fecha":"2026-09-01","estado":"Resuelto","tipo":"Vulnerabilidad"}]}
```

Figura 5. Respuesta JSON de GET `http://localhost:3000/api/incidentes`.

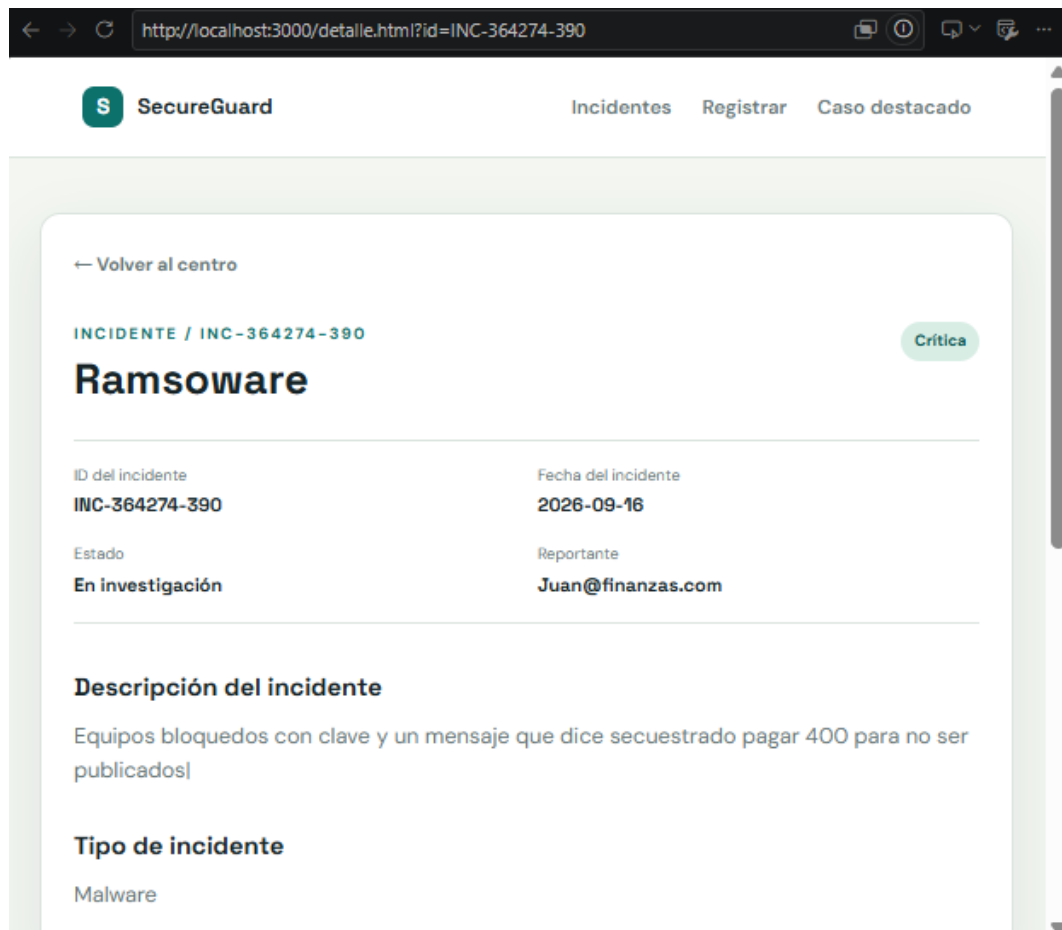


Figura 6. Detalle dinámico de INC-364274-390. La vista lee ?id= y consulta GET /api/incidentes/:id.

La Figura 6 demuestra que detalle.html ya no está fijo en INC-002. El cliente toma el parámetro de la URL, pide el recurso al servidor y pinta título, fecha, estado, reportante, descripción y tipo. En este caso: ransomware, criticidad alta, reportante Juan@finanzas.com y descripción del secuestro de equipos.

7. Cómo reproducir el prototipo

1. Tener Node.js 18 o superior (en este equipo se usó v26.4.0).
2. Abrir la carpeta SEMANA 3 en una terminal.
3. Ejecutar node server.js.
4. Visitar <http://localhost:3000> (cliente) y <http://localhost:3000/api/incidentes> (JSON).
5. Registrar un incidente y recargar: el nuevo ID debe persistir.

8. Conclusión

El entregable cumple el alcance de T1S3: define la plataforma digital, identifica componentes, servicios y roles, implementa n-capas, explica HTTP y demuestra un servidor Node.js con ruta JSON. La continuidad con SecureGuard evita un prototipo genérico y deja base para las siguientes unidades.